GRC Toolkit

Technical Documentation

Setup, Configuration & Deployment Guide

437	251	63+	35+
CONTROLS	REGO POLICIES	API ENDPOINTS	INTEGRATIONS

Contents

1. What is the GRC Toolkit?

Technology Stack

2. Quick Start

Prerequisites & Installation

3. Project Structure

4. Architecture

System Components & Request Flow

5. Compliance Frameworks

NIST 800-53 · ISO 27001 · SOC 2 · Crosswalks

6. API Reference

Auth · Dashboard · Evidence · Assessments · Risk · Frameworks · Vendors · Policies

7. Database Schema

8. Frontend Dashboard

9. Docker Deployment

10. Production Configuration

Reverse Proxy · Environment Variables · Checklist

11. Security Architecture

12. Policy-as-Code (OPA)

13. Integrations

14. Cloud Infrastructure (Terraform)

15. CI/CD Pipeline

16. Operations & Maintenance

17. Troubleshooting

18. Extending the Toolkit

1. What is the GRC Toolkit?

The GRC Toolkit is an open-source platform that helps security and compliance teams manage governance, risk, and compliance from a single system. Instead of tracking controls in spreadsheets and collecting evidence manually, the toolkit automates evidence collection across cloud providers, evaluates policies continuously, and produces audit-ready reports.

- **437 compliance controls** across NIST 800-53, ISO 27001, and SOC 2
- **Automated evidence collection** from AWS, Azure, and GCP using native APIs
- **251 OPA/Rego policies** that enforce compliance rules continuously
- **Monte Carlo risk analysis** that quantifies risk in dollars, not guesses
- **521 framework crosswalks** so you assess once and report across multiple standards
- **35+ tool integrations** including CrowdStrike, Splunk, Okta, Tenable, and more
- **Customer trust portal** for prospects to see your compliance posture
- **React dashboard** with real-time charts, trend analysis, and remediation guidance

Technology Stack

LAYER	TECHNOLOGY	VERSION
Backend	Python + FastAPI + Uvicorn	3.11+ / 0.109+
Frontend	React + TypeScript + Vite	19 / 5.9 / 8
Database	PostgreSQL (prod) / SQLite (dev)	16.2
ORM	SQLAlchemy	2.0+
Policy Engine	Open Policy Agent (OPA)	0.62.1
Styling	Tailwind CSS + Recharts	Latest
Authentication	JWT (HS256) + bcrypt	python-jose
Task Queue	Celery + Redis	5.3 / 7.2
Containers	Docker + Docker Compose	Latest
Infrastructure	Terraform	1.5+

2. Quick Start

Get the toolkit running on your machine in under five minutes. This setup uses SQLite, so you don't need PostgreSQL or Docker.

Prerequisites

- Python 3.11 or later
- Node.js 18 or later *(only if modifying the frontend)*
- Git

Step 1 — Clone & install

```
$ git clone https://github.com/your-org/grc-toolkit.git
$ cd grc-toolkit

$ python3 -m venv .venv
$ source .venv/bin/activate
$ pip install -r requirements.txt
```

Step 2 — Configure environment

```
$ cp .env.example .env
```

Open `.env` and set these values:

```
# Database (SQLite for local development)
GRC_DATABASE_URL=sqlite:///grc_toolkit.db

# JWT signing key (any random string for local dev)
GRC_SECRET_KEY=any-random-string-for-local-dev

# Leave empty to disable API key auth in dev mode
GRC_API_KEYS=
```

Step 3 — Load demo data

```
$ python -m scripts.seed_demo_data
```

This creates demo users, sample assessments for 5 frameworks, 15 integrations, 20 evidence records, 15 vendors, and 10 policy violations.

Step 4 — Start the server

```
$ uvicorn api.main:app --reload --host 0.0.0.0 --port 8000
```

Open **http://localhost:8000** in your browser.

> **Demo Credentials**
>
> **Email:** admin@grc-demo.com
>
> **Password:** demo1234

3. Project Structure

Each folder has a single responsibility. Here's where to find things and when you'd need to touch them.

FOLDER	WHAT IT CONTAINS	WHEN YOU'D TOUCH IT
`api/`	FastAPI application and middleware	Adding or modifying API endpoints
`api/routers/`	14 route files (auth, dashboard, evidence, etc.)	Adding new API features
`modules/`	Core business logic	Changing assessment, risk, or collection logic
`modules/collectors/`	AWS, Azure, GCP evidence collectors	Adding new cloud checks
`modules/connectors/`	Unified connector framework	Integrating a new security tool
`db/`	SQLAlchemy models, sessions, repositories	Changing the database schema
`config/`	YAML files for frameworks, controls, crosswalks	Adding or updating compliance controls
`policies/`	251 OPA Rego policy files	Adding or updating policy rules
`frontend/`	React TypeScript SPA source code	Changing the dashboard UI
`static/`	Built frontend assets (served by FastAPI)	After running `npm run build`
`terraform/`	IaC modules for AWS, Azure, GCP	Deploying compliance infrastructure
`tests/`	364 pytest tests	After any code changes
`scripts/`	CLI tools (seeding, assessments, exports)	Running manual operations

The full directory tree:

```
grc-toolkit/
├── api/
|   ├── main.py                 # App entry point, middleware, routes
|   ├── schemas.py              # Pydantic request/response models
|   ├── deps.py                 # Dependency injection
|   ├── security.py             # Auth, rate limiting, audit
|   └── routers/                # 14 endpoint files
├── modules/
|   ├── collectors/             # AWS, Azure, GCP collectors
|   ├── connectors/             # Unified connector framework
|   ├── control_assessor.py     # Assertion engine
|   ├── risk_engine.py          # Monte Carlo simulation
|   ├── framework_mapper.py     # Crosswalk engine
|   └── vendor_monitor.py       # Vendor risk scoring
├── db/
|   ├── models.py               # 10 SQLAlchemy tables
|   ├── session.py              # Engine & connection pooling
|   └── repository.py           # CRUD operations
├── config/
|   ├── nist_800_53.yaml        # 293 NIST controls (8,474 lines)
|   ├── iso_27001.yaml          # 93 ISO controls (2,668 lines)
|   ├── soc2.yaml               # 51 SOC 2 controls (1,972 lines)
|   ├── crosswalks.yaml         # 521 framework mappings
|   └── settings.yaml           # App configuration
├── policies/
|   ├── nist-800-53/            # 142 policies, 20 families
|   ├── iso-27001/              # 93 policies, 4 themes
|   ├── soc2/                   # 13 policies, all categories
|   └── terraform/              # 3 cloud baseline policies
├── frontend/src/               # React + TypeScript + Tailwind
├── terraform/modules/          # AWS, Azure, GCP baselines
├── tests/                      # 364 pytest tests
├── scripts/                    # CLI utilities
├── docker-compose.yaml
├── Dockerfile
└── requirements.txt
```

4. Architecture

System Components

COMPONENT	WHAT IT DOES	HOW IT CONNECTS
FastAPI App	Serves the REST API (63+ endpoints) and the React frontend	Port 8000, HTTP/HTTPS
PostgreSQL	Stores all persistent data across 10 tables	Internal port 5432
OPA Server	Evaluates Rego policies against resource data	Internal port 8181
Redis	Message broker for Celery background tasks	Internal port 6379
React SPA	Dashboard UI with 16 pages	Static files served by FastAPI

Request Flow

Every API request passes through these layers, in order:

1. **Rate Limiter** — checks if the client has exceeded their quota (120 requests/min per IP)
2. **Audit Logger** — records the request with a unique ID, timestamp, method, path, and client IP
3. **Security Headers** — adds HSTS, X-Frame-Options, and other protective headers
4. **CORS** — validates the request origin against your whitelist
5. **Authentication** — verifies the JWT token or API key
6. **Route Handler** — executes business logic and returns JSON

> **Every Request Gets Tracked**
>
> All responses include an `X-Request-Id` header for tracing. Sensitive headers (Authorization, API keys, cookies) are automatically redacted in all logs.

5. Compliance Frameworks

Control definitions live in YAML files under `config/`. The multi-file loader merges `frameworks.yaml` with individual framework files at startup. OPA Rego policies in `policies/` enforce every control.

Framework Coverage

FRAMEWORK	CONTROLS	CONFIG FILE	POLICIES
NIST SP 800-53 Rev 5	293 active controls	config/nist_800_53.yaml	142 Rego files
ISO 27001:2022 Annex A	93 controls	config/iso_27001.yaml	93 Rego files
SOC 2 Type II	51 controls	config/soc2.yaml	13 Rego files
HIPAA	64+ controls	config/frameworks.yaml	Via NIST crosswalk
CMMC Level 2	110 controls	config/frameworks.yaml	Via NIST crosswalk

NIST 800-53 — All 20 Families

ID	FAMILY	POLICIES
AC	Access Control	3
AT	Awareness and Training	4
AU	Audit and Accountability	2
CA	Security Assessment and Authorization	7
CM	Configuration Management	1
CP	Contingency Planning	9
IA	Identification and Authentication	2
IR	Incident Response	8
MA	Maintenance	6
MP	Media Protection	7
PE	Physical and Environmental Protection	17
PL	Planning	8
PM	Program Management	14
PS	Personnel Security	9
PT	PII Processing and Transparency	8
RA	Risk Assessment	7
SA	System and Services Acquisition	15
SC	System and Communications Protection	4
SI	System and Information Integrity	1
SR	Supply Chain Risk Management	10

ISO 27001:2022 — All 4 Themes

THEME	NAME	CONTROLS	POLICIES
A.5	Organizational Controls	37 (A.5.1–A.5.37)	37
A.6	People Controls	8 (A.6.1–A.6.8)	8
A.7	Physical Controls	14 (A.7.1–A.7.14)	14
A.8	Technological Controls	34 (A.8.1–A.8.34)	34

SOC 2 — All 13 Categories

CATEGORY	NAME	POLICY FILE
CC1	Control Environment	`cc1_control_environment.rego`
CC2	Communication and Information	`cc2_communication_information.rego`
CC3	Risk Assessment	`cc3_risk_assessment.rego`
CC4	Monitoring Activities	`cc4_monitoring_activities.rego`
CC5	Control Activities	`cc5_control_activities.rego`
CC6	Logical and Physical Access	`cc6_logical_access.rego`
CC7	System Operations	`cc7_system_operations.rego`
CC8	Change Management	`cc8_change_management.rego`
CC9	Risk Mitigation	`cc9_risk_mitigation.rego`
A1	Availability	`a1_availability.rego`
C1	Confidentiality	`c1_confidentiality.rego`
PI1	Processing Integrity	`pi1_processing_integrity.rego`
P1	Privacy	`p1_privacy.rego`

Framework Crosswalks

The `crosswalks.yaml` file contains **521 control mappings** across five crosswalk sections. This lets you assess against one framework and automatically map results to the others:

- NIST 800-53 → SOC 2
- NIST 800-53 → ISO 27001
- NIST 800-53 → HIPAA
- SOC 2 → ISO 27001
- ISO 27001 → NIST 800-53

6. API Reference

All endpoints live under `/api/v1/`. They return JSON and require authentication unless noted otherwise.

Authentication

METHOD	ENDPOINT	DESCRIPTION	AUTH
`POST`	`/api/v1/auth/register`	Create a new user account	None
`POST`	`/api/v1/auth/login`	Log in and receive a JWT token	None
`GET`	`/api/v1/auth/me`	Get the current user's profile	JWT
`POST`	`/api/v1/auth/logout`	Log out (invalidate client-side)	JWT

Include the token as: `Authorization: Bearer <token>`. Tokens expire after 8 hours.

Dashboard

METHOD	ENDPOINT	DESCRIPTION
`GET`	`/api/v1/dashboard/summary`	Real-time compliance posture: framework status, violations, vendor risk, integrations, recent activity

Evidence Collection

METHOD	ENDPOINT	DESCRIPTION
`POST`	`/api/v1/evidence/collect`	Trigger cloud evidence collection (returns 202 Accepted)
`GET`	`/api/v1/evidence/`	List evidence records with filtering and pagination
`GET`	`/api/v1/evidence/{id}`	Get a single evidence record with raw API data
`GET`	`/api/v1/evidence/{id}/verify`	Verify evidence integrity via SHA-256 hash

Assessments

METHOD	ENDPOINT	DESCRIPTION
`POST`	`/api/v1/assessments/run`	Start a compliance assessment (returns 202 Accepted)
`GET`	`/api/v1/assessments/runs`	List all assessment runs with status and pass rates
`GET`	`/api/v1/assessments/runs/{id}`	Get details for a specific assessment run
`GET`	`/api/v1/assessments/runs/{id}/results`	Get all check results for a run
`GET`	`/api/v1/assessments/remediation/{control_id}`	Get remediation guidance for a specific control
`GET`	`/api/v1/assessments/trend`	Compliance score trend data over time

Risk Analysis

METHOD	ENDPOINT	DESCRIPTION
`POST`	`/api/v1/risk/simulate`	Run Monte Carlo simulation for a single threat scenario
`POST`	`/api/v1/risk/portfolio`	Aggregate risk across multiple scenarios
`POST`	`/api/v1/risk/treatments`	Compare cost-effectiveness of risk treatments
`GET`	`/api/v1/risk/scenarios`	List predefined threat scenarios

Frameworks

METHOD	ENDPOINT	DESCRIPTION
`GET`	`/api/v1/frameworks/`	List all supported frameworks
`GET`	`/api/v1/frameworks/{id}`	Get framework details with control families
`GET`	`/api/v1/frameworks/{id}/controls`	Get all controls in a specific framework
`POST`	`/api/v1/frameworks/crosswalk`	Map a control to equivalents in other frameworks

Vendors

METHOD	ENDPOINT	DESCRIPTION
`POST`	`/api/v1/vendors/`	Add a new vendor
`GET`	`/api/v1/vendors/`	List vendors with pagination
`GET`	`/api/v1/vendors/dashboard`	Vendor risk dashboard with distribution chart
`GET`	`/api/v1/vendors/needing-assessment`	Vendors overdue for their next assessment
`GET`	`/api/v1/vendors/{id}`	Get a specific vendor's details
`PUT`	`/api/v1/vendors/{id}`	Update vendor (allowlisted fields only)

Policies (OPA)

METHOD	ENDPOINT	DESCRIPTION
`POST`	`/api/v1/policies/evaluate`	Evaluate a resource against OPA policies
`GET`	`/api/v1/policies/violations`	List open violations with severity filtering
`POST`	`/api/v1/policies/violations/{id}/resolve`	Mark a violation as resolved
`GET`	`/api/v1/policies/bundles`	List available policy bundles

Additional Routers

ROUTER	PREFIX	WHAT IT DOES
Integrations	`/api/v1/integrations`	Browse 35-tool catalog, connect, sync, and disconnect tools
Exports	`/api/v1/exports`	Download POA&M documents, executive summaries, audit packages
Tool Config	`/api/v1/tool-config`	Configure credentials for connected security tools
Data Silos	`/api/v1/data-silos`	Scan S3, GitHub, SharePoint for PII/PHI/credentials
Trust Portal	`/api/v1/trust`	Public compliance transparency page (no auth)
Settings	`/api/v1/settings`	Organization settings, notifications, API key management

7. Database Schema

The app uses SQLAlchemy 2.0 with 10 tables. All primary keys are UUIDs (stored as strings for SQLite/PostgreSQL portability). Tables are created automatically on startup via `init_db()`.

TABLE	WHAT IT STORES	KEY FIELDS
`users`	Login accounts	email (unique), hashed_password, role
`audit_logs`	Every data modification	actor, action, resource_type, changes (JSON)
`evidence_records`	Collected cloud evidence	control_id, provider, data (JSON), sha256_hash
`assessment_runs`	Assessment executions	framework, status, pass_rate, summary (JSON)
`assessment_results`	Individual check results	run_id (FK), control_id, status, findings (JSON)
`framework_definitions`	Framework metadata	key, version, control_count, data (JSON)
`vendor_records`	Vendor risk tracking	risk_score, risk_level, certifications (JSON)
`policy_violations`	OPA policy violations	policy_id, severity, status (open/resolved)
`asset_records`	Cloud resource inventory	provider, service, resource_type, tags (JSON)
`data_sources`	Connected data source registry	name, type, connection_status, last_sync

Key Indexes

- `evidence_records` : control_id, provider, account_id, collected_at
- `assessment_results` : run_id, control_id, status
- `users` : email (unique index)
- `policy_violations` : policy_id, status

Key Relationships

- `assessment_results.run_id` references `assessment_runs.id` (cascade delete)
- Evidence records are linked to assessment results via `evidence_ids` JSON array
- Audit logs reference any resource via `resource_type` + `resource_id` polymorphism

8. Frontend Dashboard

The frontend is a React single-page application built with TypeScript, Tailwind CSS, and Recharts. FastAPI serves the built assets as static files — no separate frontend server needed in production.

Pages

ROUTE	PAGE	WHAT IT SHOWS
`/`	Landing	Public marketing page with feature overview
`/login`	Login	Email + password authentication
`/register`	Register	New user sign-up
`/dashboard`	Dashboard	Real-time compliance posture with charts and trends
`/frameworks`	Frameworks	Browse control families and crosswalk explorer
`/assessments`	Assessments	Run assessments, view results, compliance trends
`/evidence`	Evidence	Browse and verify collected evidence records
`/risk`	Risk	Monte Carlo simulations and portfolio analysis
`/vendors`	Vendors	Vendor inventory, risk scoring, certifications
`/integrations`	Integrations	Connect and configure 35+ security tools
`/poam`	POA&M	Plan of Action and Milestones tracking
`/data-silos`	Data Silos	Data discovery and classification results
`/trust`	Trust Portal	Public compliance transparency page
`/settings`	Settings	Organization config, notifications, API keys
`/tool-config`	Tool Config	Credential management for integrations

Building the Frontend

```
$ cd frontend
$ npm install
$ npm run dev          # Dev server with hot-reload on port 5173
$ npm run build        # Production build → outputs to ../static/
```

After building, restart the FastAPI server and the new frontend is live at the root URL.

Frontend Architecture

- **React Router v6** — Client-side routing with protected route wrappers
- **TanStack Query** — Server state management with automatic caching
- **AuthContext** — JWT token management via React Context + localStorage
- **Tailwind CSS** — Utility-first styling with consistent design system
- **Recharts** — Data visualization for trends and risk analysis

9. Docker Deployment

For production, use Docker Compose to run the full stack: API, PostgreSQL, OPA, and Redis.

Services

SERVICE	IMAGE	EXPOSED PORT	HEALTH CHECK
api	Built from Dockerfile	127.0.0.1:8000	GET /health
db	postgres:16.2-alpine	Internal only	pg_isready
opa	openpolicyagent/opa:0.62.1	Internal only	GET /health
redis	redis:7.2-alpine	Internal only	redis-cli ping

Step 1 — Create environment file

```
$ cp .env.example .env
```

Step 2 — Generate secure credentials

```
# Generate a strong database password
$ python3 -c "import secrets; print(secrets.token_urlsafe(24))"

# Generate API key for programmatic access
$ python3 -c "import secrets; print(secrets.token_urlsafe(32))"

# Generate JWT signing key
$ python3 -c "import secrets; print(secrets.token_urlsafe(64))"
```

Copy the generated values into your `.env` file.

Step 3 — Launch the stack

```
$ docker compose up -d
$ docker compose ps              # Verify all services are healthy
$ docker compose logs -f api      # Tail API logs
```

Step 4 — Seed demo data (optional)

```
$ docker compose exec api python -m scripts.seed_demo_data
```

Container Security Features

- Non-root user (`grcuser`) inside all containers
- Read-only filesystem with `/tmp` tmpfs
- Database, OPA, and Redis not exposed to host network
- API bound to 127.0.0.1 only (use a reverse proxy for external access)
- Pinned image digests for supply chain security
- Health checks on all services with automatic restart

10. Production Configuration

Reverse Proxy

The API binds to localhost:8000 and should **never** be directly exposed to the internet. Put a reverse proxy in front of it.

Nginx Configuration

```
server {
    listen 443 ssl http2;
    server_name grc.example.com;

    ssl_certificate     /etc/ssl/certs/grc.crt;
    ssl_certificate_key /etc/ssl/private/grc.key;

    location / {
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

Environment Variables

VARIABLE	REQUIRED	DEFAULT	DESCRIPTION
`GRC_DATABASE_URL`	Yes	sqlite:///grc_toolkit.db	PostgreSQL connection string with sslmode=require
`GRC_SECRET_KEY`	Yes	dev-secret	JWT signing key (64+ random chars in production)
`GRC_API_KEYS`	Production	*(empty)*	Comma-separated API keys
`GRC_OPA_URL`	No	http://opa:8181	OPA server address
`GRC_CORS_ORIGINS`	No	*	Allowed CORS origins
`GRC_RATE_LIMIT_RPM`	No	120	Requests per minute per client IP
`GRC_RATE_LIMIT_BURST`	No	20	Burst request capacity
`GRC_ENABLE_DOCS`	No	false	Enable Swagger UI at /docs
`POSTGRES_PASSWORD`	Docker	—	PostgreSQL password
`REDIS_PASSWORD`	Docker	—	Redis AUTH password
`GRC_SLACK_WEBHOOK_URL`	No	—	Slack notification webhook URL
`GRC_SMTP_SERVER`	No	—	SMTP server for email alerts
`GRC_SMTP_PORT`	No	587	SMTP port
`GRC_SMTP_SENDER`	No	—	From address for alert emails

Production Checklist

- Set `GRC_API_KEYS` with strong, randomly-generated API keys
- Set `GRC_SECRET_KEY` to a cryptographically random value (64+ characters)
- Set `GRC_ENABLE_DOCS=false` to disable Swagger UI
- Set `GRC_CORS_ORIGINS` to your specific domain(s), not `*`
- Use PostgreSQL with SSL (`sslmode=require`), never SQLite in production
- Place behind a TLS-terminating reverse proxy (Nginx, Caddy, or cloud ALB)
- Configure log aggregation (CloudWatch, Datadog, ELK Stack)
- Set up automated daily database backups with tested restore procedures

- Enable cloud provider credential rotation
- Configure Slack or email notifications for critical findings

11. Security Architecture

Authentication Methods

METHOD	USED FOR	HOW IT WORKS
JWT Bearer Token	Dashboard UI, user sessions	HS256 signed, 8-hour expiry, bcrypt password hashing
API Key	Scripts, service-to-service calls	X-API-Key header, constant-time comparison, env-based
OAuth2 Password	Login endpoint	Validates credentials, returns JWT token

Security Headers

Every response includes these protective headers automatically:

HEADER	VALUE	WHAT IT PREVENTS
`X-Content-Type-Options`	nosniff	MIME-type confusion attacks
`X-Frame-Options`	DENY	Clickjacking
`X-XSS-Protection`	1; mode=block	Cross-site scripting
`Strict-Transport-Security`	max-age=31536000	HTTP downgrade attacks
`Permissions-Policy`	geolocation=(), camera=()	Unwanted browser API access

Rate Limiting

- **Algorithm:** Token bucket, per-IP tracking
- **Default:** 120 requests/min, 20 burst capacity
- **Exceeded:** Returns HTTP 429 with `Retry-After` header
- **Performance:** O(1) per request, no database queries (in-memory)

Input Validation

- All request payloads validated via Pydantic v2 schemas with type coercion
- Enum fields restricted to known-good value sets
- Vendor updates use a field allowlist to prevent mass assignment
- SQLAlchemy ORM prevents SQL injection via parameterized queries

Audit Trail

- Every request logged: timestamp, request ID, method, path, client IP, status, duration
- Sensitive headers (Authorization, API keys, cookies) redacted automatically
- Database changes tracked in `audit_logs` table with before/after JSON snapshots
- All responses include `X-Request-Id` for traceability

12. Policy-as-Code (OPA)

The toolkit uses Open Policy Agent to continuously evaluate your infrastructure against compliance rules written in Rego. With 251 policy files, every control across all three major frameworks has enforcement coverage.

How It Works

1. Client sends resource data to `POST /api/v1/policies/evaluate`
2. API forwards the data to OPA at `http://opa:8181`
3. OPA evaluates the matching Rego policies against the resource data
4. Violations are returned and stored in the `policy_violations` table
5. Unresolved violations appear on the dashboard and trigger Slack/email alerts

Policy Organization

```
policies/
├── nist-800-53/                # 142 policies across 20 families
|   ├── ac/                     # Access Control
|   ├── at/                     # Awareness and Training
|   ├── au/                     # Audit and Accountability
|   ├── ca/                     # Security Assessment
|   ├── cm/                     # Configuration Management
|   ├── cp/                     # Contingency Planning
|   ├── ia/                     # Identification & Auth
|   ├── ir/                     # Incident Response
|   ├── ma/  mp/  pe/  pl/      # Maintenance, Media, Physical, Planning
|   ├── pm/  ps/  pt/           # Program Mgmt, Personnel, PII
|   ├── ra/  sa/  sc/  si/  sr/ # Risk, Acquisition, Comms, Integrity, Supply Chain
├── iso-27001/                  # 93 policies across 4 themes
|   ├── a5/                     # Organizational (37 policies)
|   ├── a6/                     # People (8 policies)
|   ├── a7/                     # Physical (14 policies)
|   └── a8/                     # Technological (34 policies)
├── soc2/                       # 13 policies (all categories)
└── terraform/                  # 3 cloud baseline policies
```

Terraform Policy Validation

Validate Terraform plans against compliance policies before applying:

```
$ terraform plan -out=plan.tfplan
$ terraform show -json plan.tfplan > plan.json
$ conftest test plan.json -p policies/terraform/
```

13. Integrations

The toolkit connects to 35+ security tools across 9 categories. Each integration maps to the compliance controls it helps satisfy.

CATEGORY	TOOLS	CONTROLS
Cloud Providers	AWS, Azure, GCP, Prisma Cloud	AC, AU, CM, SC, SI
EDR / Endpoints	CrowdStrike, Defender, SentinelOne	SI-3, SI-4, IR-4
Identity & Access	Okta, Entra ID, CyberArk, SailPoint	AC-2, AC-6, IA-2, IA-5
Vulnerability Scanners	Tenable, Qualys, Rapid7, Wiz	RA-5, SI-2, CM-6
SIEM / Monitoring	Sentinel, Splunk, Elastic, Datadog	AU-6, SI-4, IR-4, IR-6
DevSecOps	GitHub, GitLab, Snyk	SA-11, CM-3, SI-2
GRC Platforms	ServiceNow, Drata, Vanta	CA-2, CA-7, PM-9
Alerting	PagerDuty, Slack, Jira	IR-4, IR-6, IR-7
AI / LLM	OpenAI, Anthropic, Gemini, Ollama	Custom analysis

Configure credentials through the **Tool Config** page in the dashboard or via the `/api/v1/tool-config` API. For production, consider injecting credentials via a secrets manager (AWS Secrets Manager, HashiCorp Vault, Azure Key Vault).

14. Cloud Infrastructure (Terraform)

These Terraform modules deploy compliance-enforcing infrastructure baselines that the toolkit then monitors.

AWS Modules

MODULE	WHAT IT CREATES	CONTROLS
`secure-account-baseline`	CloudTrail, GuardDuty, Security Hub, SNS alerts	AU-2, AU-3, AU-6, SI-4, IR-6
`compliant-vpc`	VPC, subnets, flow logs, NAT, default-deny SGs	SC-7, AC-4, AU-2, SC-8
`iam-baseline`	Auditor role, root account alerts, MFA enforcement	AC-2, AC-6, IA-2, IA-5

Azure & GCP Modules

MODULE	WHAT IT CREATES	CONTROLS
`azure/secure-subscription-baseline`	Log Analytics, Activity Log diagnostics, storage encryption	AU-2, AU-3, SC-28
`gcp/secure-project-baseline`	Audit log sink, org policy constraints, OS login	AU-2, AC-2, CM-6

Usage

```
$ cd terraform/modules/aws/secure-account-baseline
$ terraform init
$ terraform plan -out=plan.tfplan
$ terraform apply plan.tfplan
```

Review `variables.tf` in each module to customize regions, notification emails, and retention periods.

15. CI/CD Pipeline

GitHub Actions runs these quality gates on every push and pull request. All must pass before merging.

CHECK	TOOL	GATE CRITERIA
Linting	`ruff check`	Zero violations
Type checking	`mypy`	No type errors
Unit tests	`pytest` + coverage	All pass, ≥60% coverage
Security analysis	`bandit` (SAST)	No high/critical findings
Dependency audit	`pip-audit`	No known CVEs
Container scan	`trivy`	No critical vulnerabilities
Secret detection	`gitleaks`	No leaked credentials
Policy tests	`opa test -v`	All Rego tests pass
Policy formatting	`opa fmt --diff`	Consistent formatting
Terraform validation	`terraform validate`	Valid HCL syntax
Terraform compliance	`conftest verify`	Plans pass policies

Compliance Gate

A separate `compliance-gate.yaml` workflow blocks PRs that modify `terraform/` or `policies/` directories until all security checks pass **and** a policy review approval is obtained.

16. Operations & Maintenance

Health Monitoring

```
$ curl http://localhost:8000/health
{"status": "healthy"}
```

Use this endpoint for load balancer health checks and uptime monitoring.

Database Backups

```
# Create a backup
$ docker compose exec db pg_dump -U grc grc_toolkit > backup_$(date +%Y%m%d).sql

# Restore from backup
$ docker compose exec -T db psql -U grc grc_toolkit < backup_20260314.sql
```

Scaling

- **Horizontal:** Run multiple API containers behind a load balancer. The app is stateless (JWT + database).
- **Database:** Use managed PostgreSQL (RDS, Cloud SQL) with read replicas for dashboard queries.
- **Workers:** Scale Celery workers independently for evidence collection and assessment jobs.
- **Storage:** Configure S3/GCS/Azure Blob for evidence files in large deployments.

CLI Scripts

SCRIPT	WHAT IT DOES	COMMAND
`seed_demo_data.py`	Load demo data	`python -m scripts.seed_demo_data`
`run_assessment.py`	Run an assessment	`python scripts/run_assessment.py --framework nist_800_53`
`run_collection.py`	Collect evidence	`python scripts/run_collection.py --provider aws`
`run_risk_analysis.py`	Monte Carlo sim	`python scripts/run_risk_analysis.py --scenarios data_breach`
`generate_demo_exports.py`	Sample reports	`python scripts/generate_demo_exports.py`

> **Important**
>
> Always run scripts from the project root directory with the virtual environment activated, or use `python -m scripts.<name>` syntax.

17. Troubleshooting

PROBLEM	CAUSE	FIX
`ModuleNotFoundError: api`	Running from wrong directory	`cd` to project root, use `python -m scripts.<name>`
`.venv/bin/python: no such file`	Not in the project directory	`cd /path/to/grc-toolkit` first
Connection refused on :8000	Server not running	`uvicorn api.main:app --reload`
401 Unauthorized	Expired or missing JWT	Log in again at `/api/v1/auth/login`
OPA connection failed	OPA container not running	`docker compose up -d opa`
Database locked (SQLite)	Multiple writers	Switch to PostgreSQL
`bcrypt` import error	Missing dependency	`pip install passlib[bcrypt]`
Tests fail on Python 3.9	Uses `datetime.UTC`	Upgrade to Python 3.11+
Wrong demo credentials	Seed data not loaded	`python -m scripts.seed_demo_data`

Debug Mode

Enable verbose logging and the interactive API explorer:

```
$ GRC_ENABLE_DOCS=true uvicorn api.main:app --reload --log-level debug

# Then open http://localhost:8000/docs for the Swagger UI
```

18. Extending the Toolkit

Add a New Control Assertion

1. Define the check in `config/frameworks.yaml` under the appropriate control
2. Write the assertion function in `modules/control_assessor.py`
3. Create a Rego policy in `policies/<framework>/<family>/`
4. Add test coverage in `tests/`

Add a New Connector

1. Subclass `BaseConnector` in `modules/connectors/`
2. Implement `validate_config()`, `collect()`, and `health_check()`
3. Register with `ConnectorRegistry`
4. Add credential fields to `api/routers/tool_config.py`

Add a New API Router

1. Create the router file in `api/routers/<name>.py`
2. Import and register in `api/main.py` with `app.include_router()`
3. Add Pydantic schemas in `api/schemas.py`
4. Add route tests in `tests/test_api.py`

Add a New Framework

1. Create `config/<framework_key>.yaml` with control definitions
2. Add crosswalk entries to `config/crosswalks.yaml`
3. Create Rego policies in `policies/<framework>/`
4. Add assertions in `modules/control_assessor.py`
5. Add test coverage

End of Document — GRC Toolkit v0.3.0 — March 2026